



COMPUTER NETWORKS – ENCS3320

Project I - Socket Programming

Prepared by:

Qosai Badaha	1210469	Section: 2
Ziad Ikhraiwesh	1222671	Section 1
Yamen Kharoub	1220432	Section 1

Date: 11/5/2025

Table of Contents

Task 1 – Network Commands and Wireshark:	5
Task 2 – Implement a Simple Web Server Using Socket Programming	13
Task 3 - TCP/UDP Hybrid Client-Server Game Using Socket Programming	35
References:.....	41

Table of Figures

Figure 1ipconfig /all command	6
Figure 2: Ping command for a device within the same local network.....	6
Figure 3Ping command for discover gaia.cs.umass.edu	7
Figure 4Tracert command for discover gaia.cs.umass.edu	7
Figure 5nslookup command for discover gaia.cs.umass.edu	8
Figure 6Talent command for discover gaia.cs.umass.edu	8
Figure 7: web divs	13
Figure 8 main_en divs	15
Figure 9 : main_en divs	15
Figure 10 : main_en divs	16
Figure 11: main_en divs	16
Figure 12: main_en divs	17
Figure 13: main_en divs	17
Figure 14 : Form photo	18
Figure 15: form html code	18
Figure 16: JS form code.....	19
Figure 17: JS form code.....	19
Figure 18: JS form code.....	19
Figure 19: main_ar page	20
Figure 20: mySite_2671_ar page	20
Figure 21: / req.....	21
Figure 22:/en req	22
Figure 23: index.html req.....	22
Figure 24: main_en.html req	23
Figure 25: ar req.....	23
Figure 26: main_ar.html req	24
Figure 27: fail request	24
Figure 28: initiate server	25
Figure 29: handling req msg for the server.....	25
Figure 30: create response msg for the server	26
Figure 31 HTTP request.....	27
Figure 32: form page functionillity.....	28
Figure 33: 307 Temporary Redirect.....	30
Figure 34: Temporary redirect request	30
Figure 35: Temporary redirect request.....	32
Figure 36	35
Figure 37	35
Figure 38	35
Figure 39	36
Figure 40	36
Figure 41	36
Figure 42	37
Figure 43	37

Figure 44	38
Figure 45	38
Figure 46	39
Figure 47	39

Task 1 – Network Commands and Wireshark:

A) Briefly explain each of the following commands in your own words, using no more than two sentences per command:

(i) Ipconfig: ipconfig command is used to display the network configuration details of your computer. It shows important information such as your computer's IP address, subnet mask, default gateway, and DNS server addresses, helping troubleshoot or manage network settings. Adding /all provides even more details like the MAC address and the DHCP lease information.

(ii) Ping: ping command is used to troubleshoot the connection and reachability by sending 4 requests to a server to see if it's online and how fast it responds. It shows us how many seconds each request takes to reach the server.

(iii) Tracert: tracert command is used to display the time and route of a packet travelling between the computer and a server following the trail of messages to see all the stops they make.

(iv) Telnet: telnet command is used to remotely control and communicate with another server as if you were using it directly. It's mainly used to check whether the server is responding or not.

(v) Nslookup: nslookup command is used to find the corresponding IP address or DNS of a domain name. It also can be used to do the opposite.

complete the following steps:

1. Execute the `ipconfig /all` command on your computer and locate the IP address, subnet mask, default gateway, and DNS server addresses for your main network interface.

```
Connection-specific DNS Suffix . :  
Description . . . . . : Intel(R) Dual Band Wireless-AC 8260  
Physical Address. . . . . : B8-8A-60-DC-C2-01  
DHCP Enabled. . . . . : Yes  
Autoconfiguration Enabled . . . . : Yes  
Link-local IPv6 Address . . . . . : fe80::d4ef:28f6:662a:ed4e%9(Preferred)  
IPv4 Address. . . . . : 192.168.1.22(Preferred)  
Subnet Mask . . . . . : 255.255.255.0  
Lease Obtained. . . . . : Thursday, May 8, 2025 12:41:38 PM  
Lease Expires . . . . . : Friday, May 9, 2025 12:41:49 PM  
Default Gateway . . . . . : 192.168.1.1  
DHCP Server . . . . . : 192.168.1.1  
DHCPv6 IAID . . . . . : 112757344  
DHCPv6 Client DUID. . . . . : 00-01-00-01-2E-B6-7A-E7-C8-5B-76-7F-41-11  
DNS Servers . . . . . : 192.168.1.1  
NetBIOS over Tcpip. . . . . : Enabled
```

Figure 1: ipconfig /all command

2. Send a ping request to a device within your local network, such as from a laptop to a smartphone connected to the same wireless network.

```
C:\Users\Lenovo>ping 192.168.1.4  
  
Pinging 192.168.1.4 with 32 bytes of data:  
Reply from 192.168.1.4: bytes=32 time=124ms TTL=64  
Reply from 192.168.1.4: bytes=32 time=27ms TTL=64  
Reply from 192.168.1.4: bytes=32 time=45ms TTL=64  
Reply from 192.168.1.4: bytes=32 time=42ms TTL=64  
  
Ping statistics for 192.168.1.4:  
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),  
Approximate round trip times in milli-seconds:  
    Minimum = 27ms, Maximum = 124ms, Average = 59ms
```

Figure 2: Ping command for a device within the same local network

3. Ping `gaia.cs.umass.edu`, and using the results, provide a brief explanation of whether you believe the response originates from.

```
C:\Users\Lenovo>ping gaia.cs.umass.edu

Pinging gaia.cs.umass.edu [128.119.245.12] with 32 bytes of data:
Reply from 128.119.245.12: bytes=32 time=139ms TTL=38
Reply from 128.119.245.12: bytes=32 time=145ms TTL=38
Reply from 128.119.245.12: bytes=32 time=294ms TTL=38
Reply from 128.119.245.12: bytes=32 time=292ms TTL=38
```

Figure 3 Ping command for discover `gaia.cs.umass.edu`

Based on the results, it is clear that the response is coming from a remote external server, specifically from the address `128.119.245.12`. The TTL value of 38 indicates that there are several routers between my device and the server, suggesting that it is not on the same local network. Additionally, the relatively high response time (ranging from 139ms to 294ms) further supports the conclusion that the server is located in a geographically distant location.

4. Run `tracert/traceroute gaia.cs.umass.edu` to see the route it takes.

```
C:\Users\Lenovo>tracert gaia.cs.umass.edu

Tracing route to gaia.cs.umass.edu [128.119.245.12]
over a maximum of 30 hops:

  0  3 ms   9 ms   1 ms  192.168.1.1
  1  4 ms   6 ms   5 ms  10.74.32.250
  2  53 ms  64 ms  51 ms  10.74.19.29
  3  58 ms  52 ms  52 ms  10.74.19.150
  4  55 ms  56 ms  71 ms  10.74.59.190
  5  78 ms  47 ms  46 ms  et-4-0-13.edge1.Marseille3.Level3.net [213.242.111.121]
  6  63 ms  62 ms  60 ms  ae1.3505.edge4.mrs1.neo.colt.net [171.75.8.243]
  7  *      *      *      Request timed out.
  8  64 ms  62 ms  62 ms  be2779.ccr41.par01.atlas.cogentco.com [154.54.72.109]
  9  68 ms  104 ms 71 ms  be3684.ccr51.lhr01.atlas.cogentco.com [154.54.60.170]
 10 144 ms 141 ms 140 ms be3393.ccr31.bos01.atlas.cogentco.com [154.54.47.141]
 11 141 ms 136 ms 142 ms be8038.rcr71.orh02.atlas.cogentco.com [154.54.169.254]
 12 138 ms 141 ms 138 ms be8628.rcr51.orh01.atlas.cogentco.com [154.54.164.126]
 13 142 ms 146 ms 141 ms 38.104.218.14
 14 145 ms 145 ms 147 ms 69.16.0.8
 15 141 ms 143 ms 138 ms 69.16.1.0
 16 141 ms 142 ms 143 ms core2-rt-et-8-3-0.gw.umass.edu [192.80.83.113]
 17 150 ms 146 ms 155 ms n1-rt-1-1-et-10-0-0.gw.umass.edu [128.119.0.120]
 18 149 ms 149 ms 150 ms n1-fnt-fw-1-1-1-31-v11092.gw.umass.edu [128.119.77.233]
 19 *      *      *      Request timed out.
 20 147 ms 145 ms 145 ms core1-rt-et-7-2-1.gw.umass.edu [128.119.0.217]
 21 162 ms 148 ms 178 ms n5-rt-1-1-xe-2-1-0.gw.umass.edu [128.119.3.33]
 22 147 ms 145 ms 146 ms cics-rt-xe-0-0-0.gw.umass.edu [128.119.3.32]
 23 *      *      *      Request timed out.
 24 153 ms 139 ms 140 ms gaia.cs.umass.edu [128.119.245.12]

Trace complete.
```

Figure 4 Tracert command for discover `gaia.cs.umass.edu`

5. Execute the nslookup command to retrieve the Domain Name System (DNS) information for gaia.cs.umass.edu.

```
C:\Users\Lenovo>nslookup gaia.cs.umass.edu
Server: UnKnown
Address: 192.168.1.1

Non-authoritative answer:
Name: gaia.cs.umass.edu
Address: 128.119.245.12
```

Figure 5 nslookup command for discover gaia.cs.umass.edu

6. Use telnet to try connecting to gaia.cs.umass.edu at port 80.

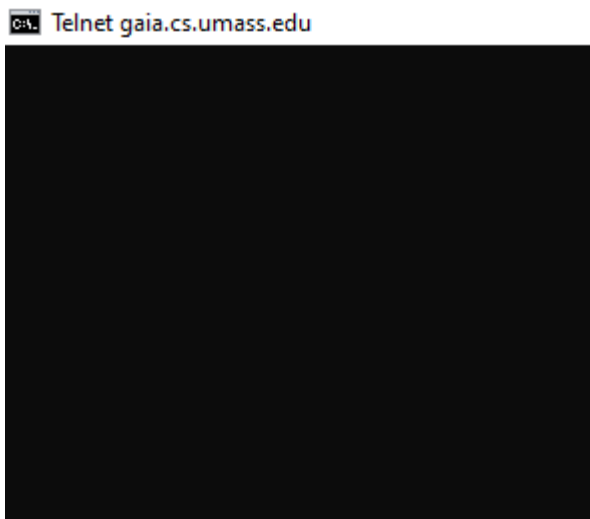


Figure 6 Talent command for discover gaia.cs.umass.edu

7. Provide details about the autonomous system (AS) number, number of IP addresses, prefixes, peers, and the name of Tier 1 ISP(s) associated with gaia.cs.umass.edu.

AS Number:

AS1249

IP Address Range:

The prefix **128.119.0.0/16** contains approximately **65,536 IPv4 addresses**.

Peers:

- **AS1968 – UMASSNET**
- **AS168 – University of Massachusetts - AMHERST**

Tier 1 ISP:

No direct Tier 1 ISP listed.

A) Use Wireshark to capture a DNS query and response for `gaia.cs.umass.edu` (make sure that you clear your cache before request this website).

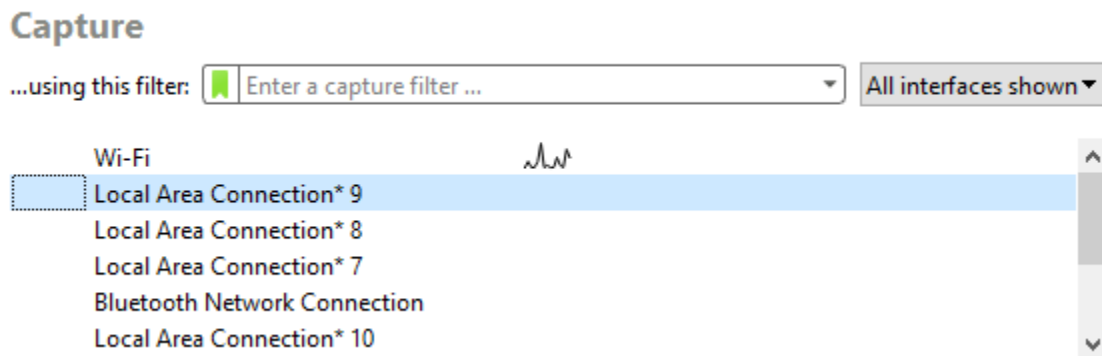
1. Objective of the Experiment:

Analyze how the computer sends a DNS query to obtain the IP address of the site `gaia.cs.umass.edu` using Wireshark, and confirm that the DNS response is received from the DNS server.

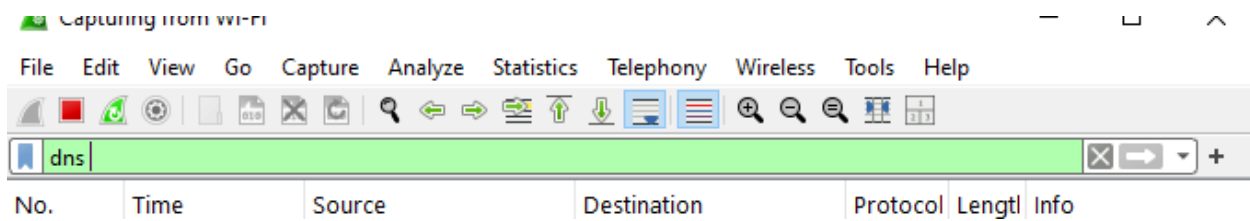
2. Steps Taken:

1. I flushed the DNS cache using the command:
`ipconfig /flushdns`

2. I started Wireshark and selected the appropriate network interface (Wi-Fi).



3. I began the capture process using the filter:
`dns`

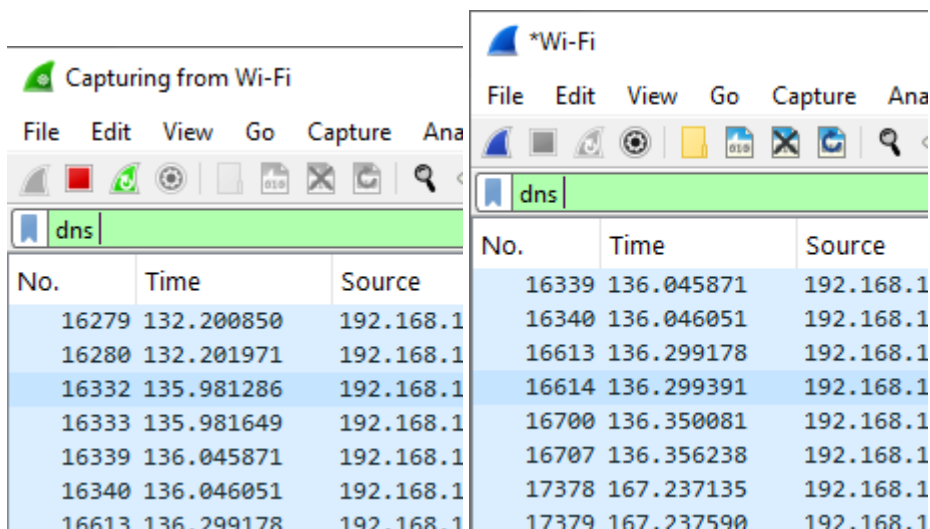


4. I opened the browser and visited the site:

<http://gaia.cs.umass.edu>



5. I stopped the capture after the DNS query-related packets appeared.



3. Results:

1.1	DNS	77	Standard query 0x1c88 HTTPS gaia.cs.umass.edu
1.1	DNS	83	Standard query 0x1460 A safebrowsing.google.com
1.1	DNS	83	Standard query 0xe5ff HTTPS safebrowsing.google.com
1.14	DNS	351	Standard query response 0x901a A optimizationguide-pa.googlea...
1.14	DNS	155	Standard query response 0xef46 HTTPS optimizationguide-pa.goo...
1.14	DNS	93	Standard query response 0xde32 A gaia.cs.umass.edu A 128.119....
1.14	DNS	130	Standard query response 0x1c88 HTTPS gaia.cs.umass.edu SOA un...

A packet appeared containing:

Standard query 0x1c88 HTTPS gaia.cs.umass.edu

This indicates that the computer sent a DNS query to obtain the IP address.

Immediately after, another packet appeared:

Standard query response 0x1c88 HTTPS gaia.cs.umass.edu

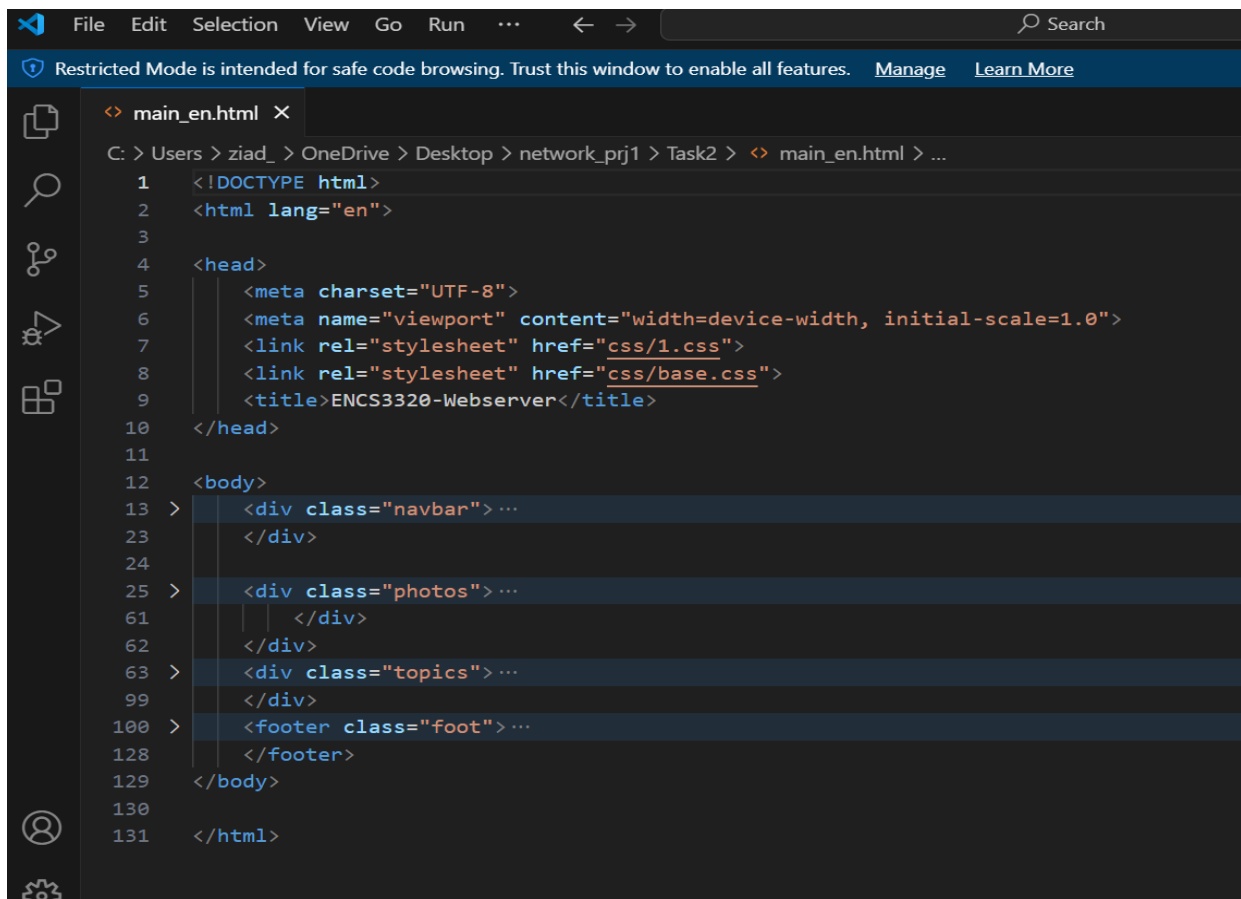
This is the response from the DNS server, containing the IP address associated with the site.

The matching query ID (0x1c88) in both the request and the response confirms that the response corresponds to the same request.

Task 2 – Implement a Simple Web Server Using Socket Programming

Part A: web design

for our web&server design we use HTML, CSS, PYTHON, JS languages for the web pages main_en.html and main_ar.html design we divide this pages into four div in html shown in the photo:



```
1 <!DOCTYPE html>
2 <html lang="en">
3
4 <head>
5     <meta charset="UTF-8">
6     <meta name="viewport" content="width=device-width, initial-scale=1.0">
7     <link rel="stylesheet" href="css/1.css">
8     <link rel="stylesheet" href="css/base.css">
9     <title>ENCS3320-Webserver</title>
10 </head>
11
12 <body>
13 > <div class="navbar"> ...
23 </div>
24
25 > <div class="photos"> ...
61 </div>
62 </div>
63 > <div class="topics"> ...
99 </div>
100 > <footer class="foot"> ...
128 </footer>
129 </body>
130
131 </html>
```

Figure 7: web divs

And we have four html pages according to the project

1) main_en.html

2)main_ar.html

3)mySite_2671_en.html

4)mySite_2671_ar.html

main_en.html page design

This page has four dives according to the project requirements which is

- 1) navbar div: to have the sentence Welcome to ENCS3320
- 2) photos div: Team members' photos (in .jpg & png format) displayed alongside their names and IDs, arranged in a table-like structure. Organize each member's information within separate boxes on the page.
- 3) topics div: A well-presented topic from the first two chapters of the textbook, including elements such as paragraphs, images (in .jpg format), and ordered and unordered lists. Use appropriate headings, font styles, colors, and text formatting (bold/italic).
- 4) footer div: to determine the end of our simple page and have the links that the project requested it P.S. for the second div we love to add our touch which is to the details of each member of the team you need just to put the pointer of the mouse on the photo and the details will be shown by itself.

And here are photos for our web page:

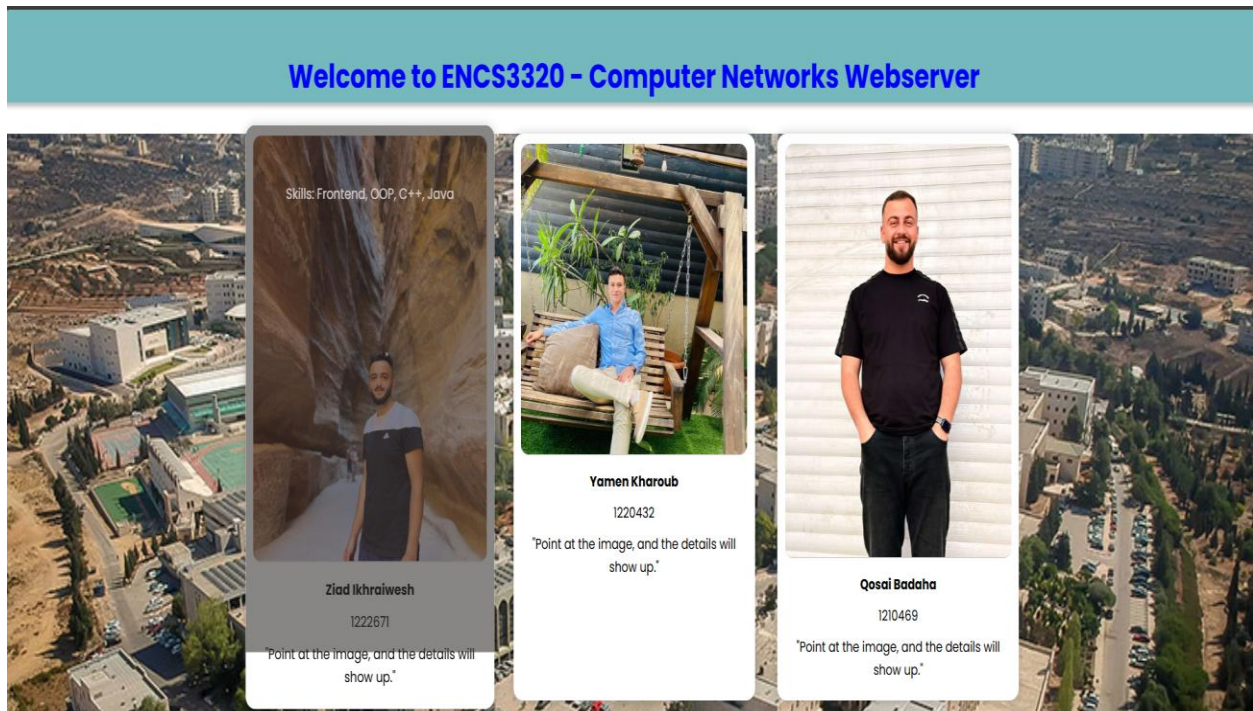


Figure 8 main_en divs

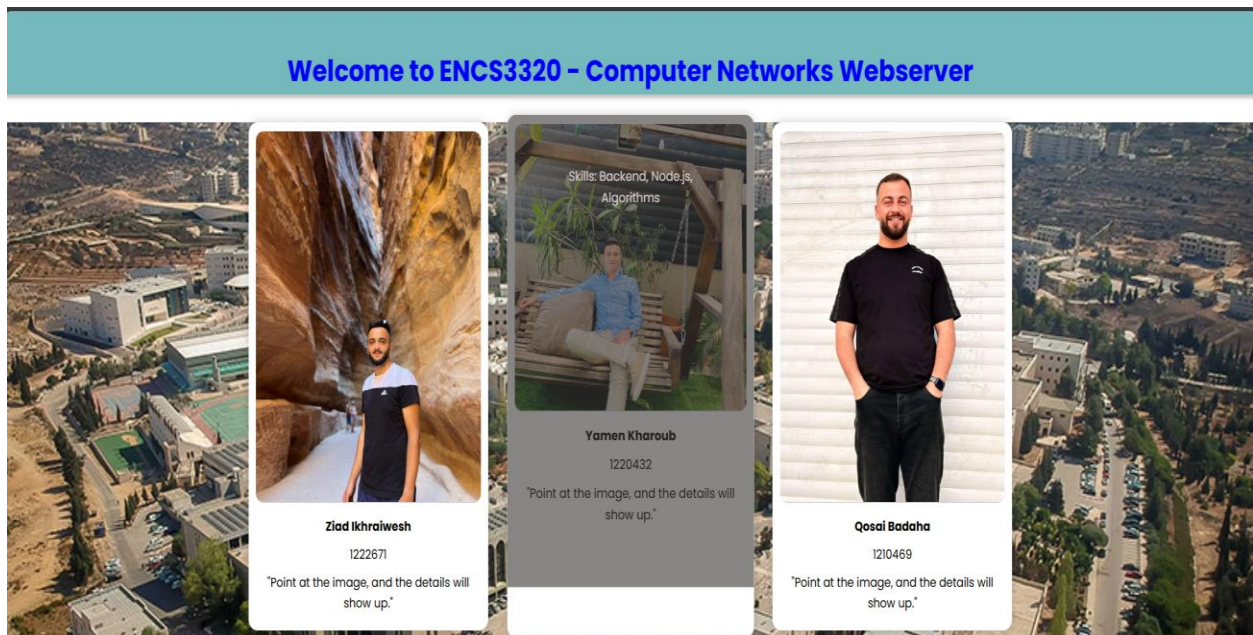


Figure 9 : main_en divs

Welcome to ENCS3320 – Computer Networks Webserver

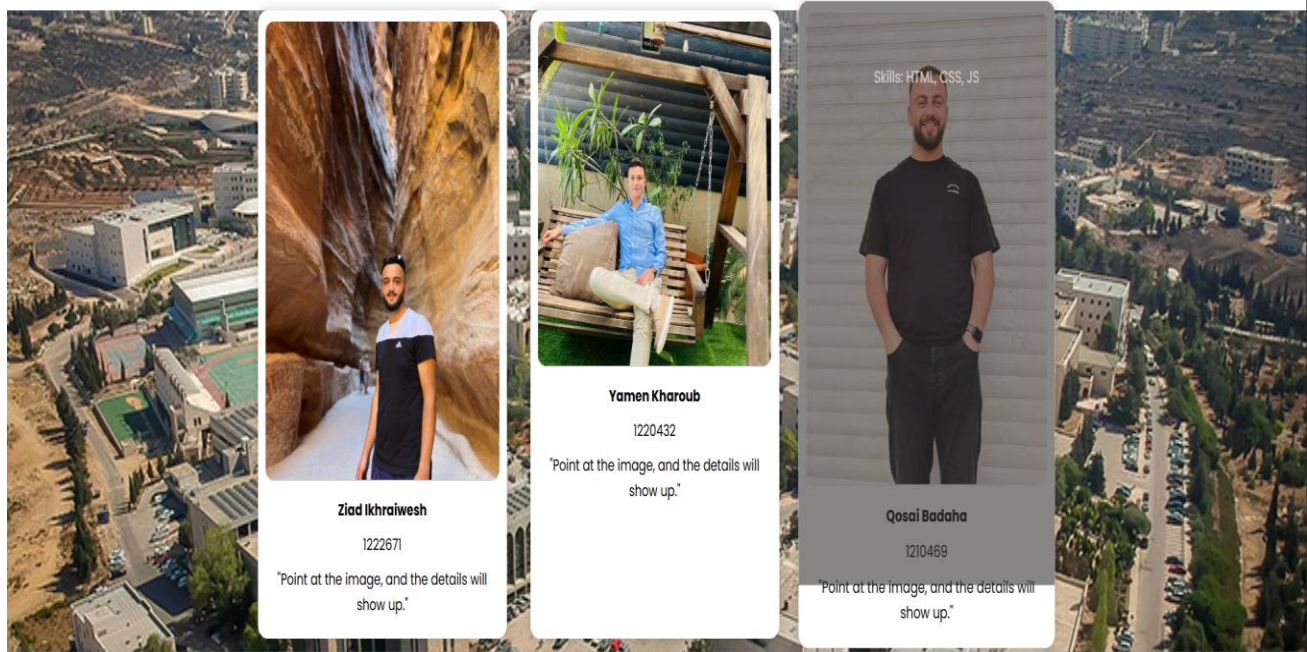


Figure 10 : main_en divs

Welcome to ENCS3320 – Computer Networks Webserver

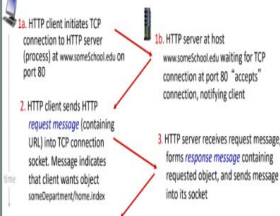
HTTP connections: two types

Non-persistent HTTP

1. TCP connection opened
2. at most one object sent over TCP connection
3. TCP connection closed
4. downloading multiple objects required multiple connections

Non-persistent HTTP: example

User enters URL: `www.someschool.edu/someDepartment/home_index`
(containing text, references to 10 jpeg images)



Non-persistent HTTP: example (cont.)

User enters URL: `www.someschool.edu/someDepartment/home_index`

persistent HTTP

- $\$$ TCP connection opened to a server
- $\$$ multiple objects can be sent over single TCP connection between client, and that server
- $\$$ TCP connection closed

Persistent HTTP

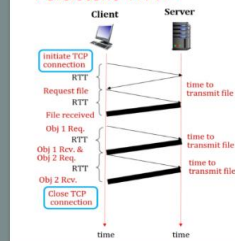


Figure 11: main_en divs

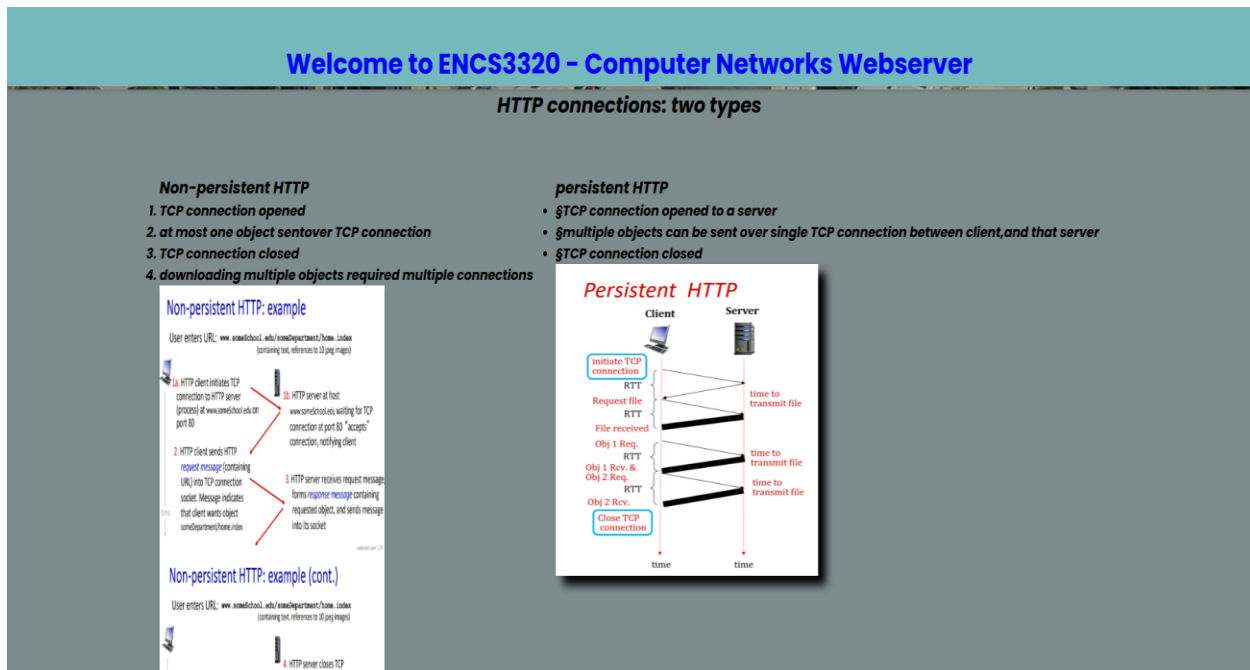


Figure 12: main_en divs

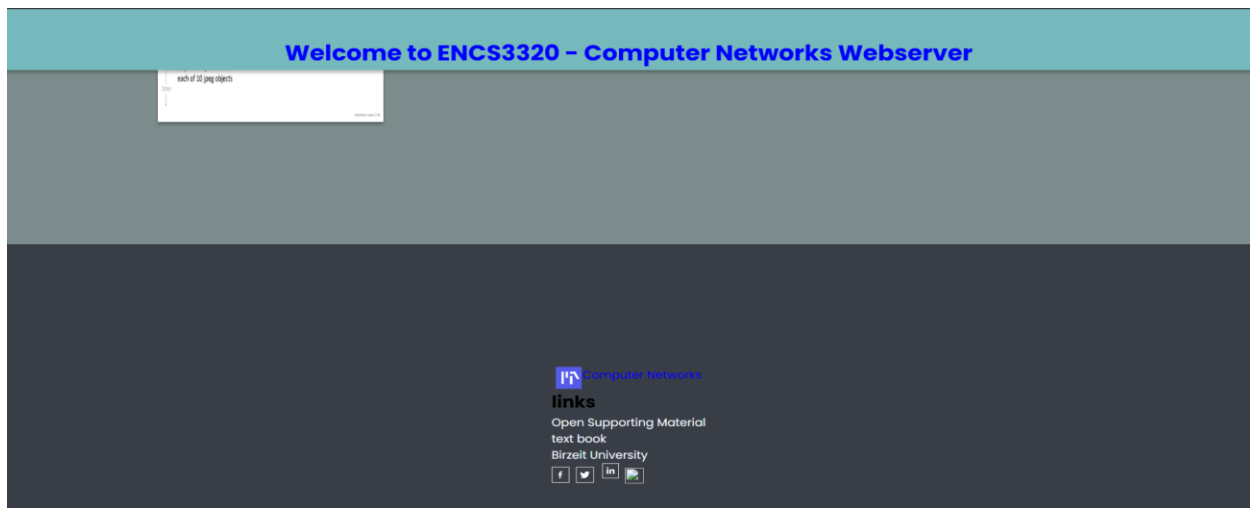


Figure 13: main_en divs

In the fourth div we have links to html page mySite_2671_en.html
 And link to text book and link to Birzeit University website and we have four icons
 which is Facebook, twitter, LinkedIn, Instagram and there is not just an icon if you
 click on any of them it will lead you to the birzeit university accounts on their
 websites.

mySite_2671_en page

this page we can reach by click on the link (Open Supporting Material) in main_en.html page and this have form to enter the name of the photo or the video that we want to reach it

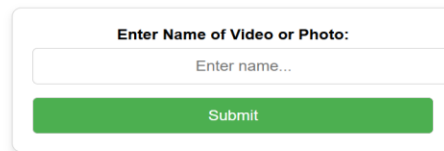


Figure 14 : Form photo

And this photo has two parts of code the first part is for HTML and CSS design

```
main_en.html  mySite_2671_en.html X
C:\Users\ziad\OneDrive\Desktop\network_prj1\Task2> mySite_2671_en.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4    <meta charset="UTF-8">
5    <meta name="viewport" content="width=device-width, initial-scale=1.0">
6    <title>Video or Photo Form</title>
7    <link rel="stylesheet" href="css/sup.css">
8  </head>
9  <body>
10   <div id="form_container">
11     <form method="get" action="/">
12       <div class="form-group">
13         <label for="file">Enter Name of Video or Photo:</label>
14         <input type="text" id="file" name="file" placeholder="Enter name..." required>
15       </div>
16       <button type="submit">Submit</button>
17     </form>
18   </div>
19
20   <!-- Dynamically display the image or video based on the input -->
21   <div id="media_display">
22     <img id="image" src="" alt="Requested Image" style="display: none;" />
23     <video id="video" controls style="display: none;">
24       <source src="" type="video/mp4">
25       Your browser does not support the video tag.
26     </video>
27   </div>
28
```

Figure 15: form html code

And the second part is for JS code to related the html page to python server code

```
29 <script>
30 document.querySelector("form").addEventListener("submit", function(e) {
31     e.preventDefault(); // Prevent form from submitting normally
32
33     var mediaName = document.getElementById("file").value;
34     var image = document.getElementById("image");
35     var video = document.getElementById("video");
36
37     // Hide both image and video initially
38     image.style.display = "none";
39     video.style.display = "none";
40
41     // Check if it's a video or an image
42     if (mediaName.endsWith(".jpg") || mediaName.endsWith(".png")) {
43         image.style.display = "block";
44         image.src = mediaName; // Set image source
45     } else if (mediaName.endsWith(".mp4")) {
46         video.style.display = "block";
47         video.querySelector("source").src = mediaName; // Set video source
48         video.load(); // Load the video
49     } else {
50         alert("Unsupported media type");
51     }
52
53     // Send a request to check if the media exists and handle redirects
54     fetch("/") + mediaName, {
```

Figure 16: JS form code

```
53 // Send a request to check if the media exists and handle redirects
54 fetch("/") + mediaName, {
55     method: "GET"
56 }
57 .then(response => {
58     if (response.status === 307) {
59         // Redirect automatically if status is 307
60         const redirectUrl = response.headers.get("Location");
61         window.location.href = redirectUrl; // Perform the redirect
62     } else {
63         return response.blob(); // If not a redirect, get the response body
64     }
65 })
66 .then(blob => {
67     if (blob) {
68         // If we have a valid image or video, update the media display
69         const contentType = blob.type;
70         if (contentType.startsWith("image")) {
71             // Image response
72             const objectURL = URL.createObjectURL(blob);
73             image.style.display = "block";
74             image.src = objectURL;
75         } else if (contentType.startsWith("video")) {
76             // Video response
```

Figure 17: JS form code

```
72     const objectURL = URL.createObjectURL(blob);
73     image.style.display = "block";
74     image.src = objectURL;
75 } else if (contentType.startsWith("video")) {
76     // Video response
77     const objectURL = URL.createObjectURL(blob);
78     video.style.display = "block";
79     video.querySelector("source").src = objectURL;
80     video.load();
81 }
82 }
83 })
84 .catch(error => {
85     console.error("Error:", error);
86 });
87 });
88 </script>
```

Figure 18: JS form code

P.S. the main_ar.html and mySite_2671_ar.html pages have the same design
And here are some photos for these pages.

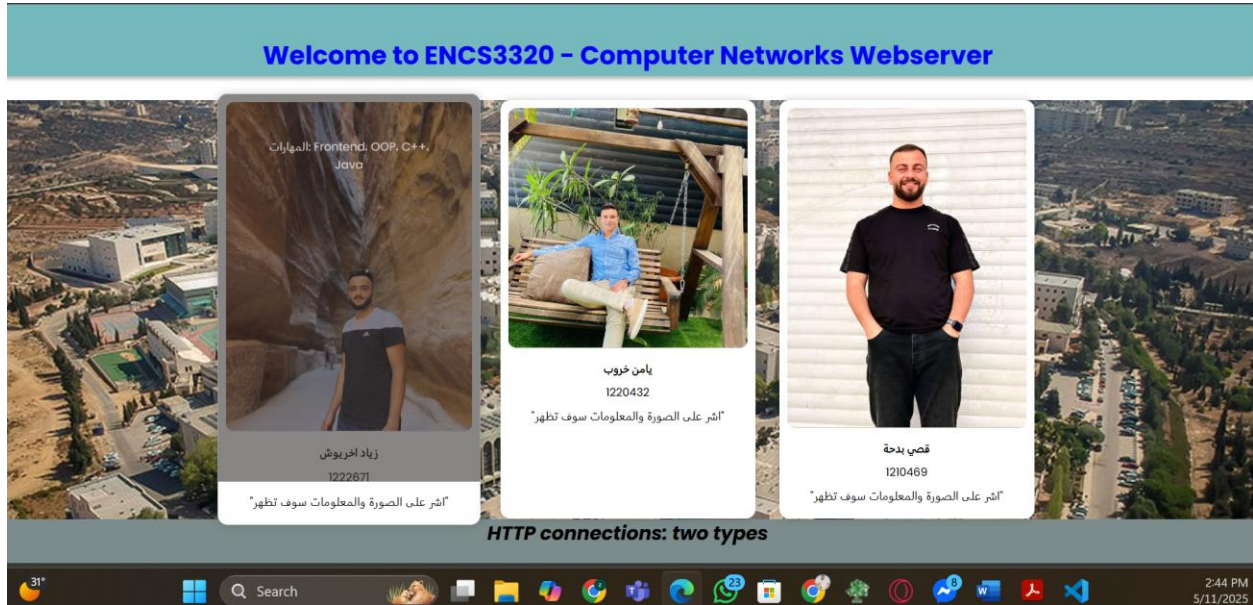


Figure 19: main_ar page

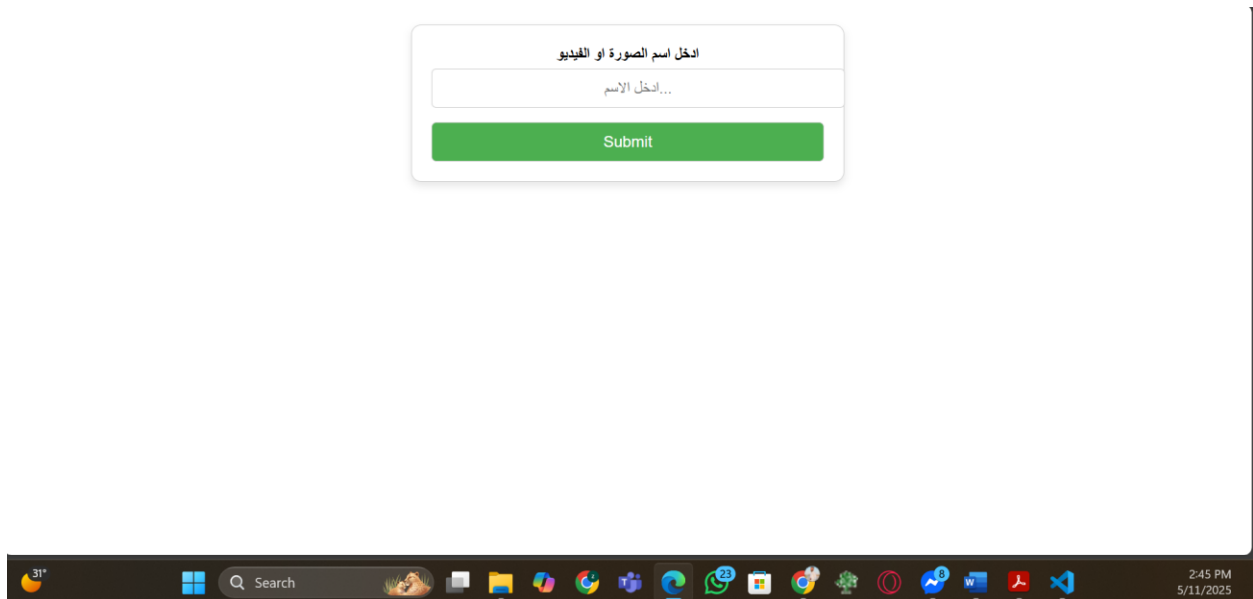


Figure 20: mySite_2671_ar page

Part B: Server functionality

In part B, the request is '/' or '/index.html' or '/main_en.html' or '/en', so the server sends main_en.html file with content-Type: text/html; charset=UTF-8.

In part B, the request is '/ar' or '/main_ar.html', so the server sends main_ar.html file with content-Type: text/html; charset=UTF-8.

P.S. For the request "/", the browser automatically removes the "/" and sends me to the main page as normal.

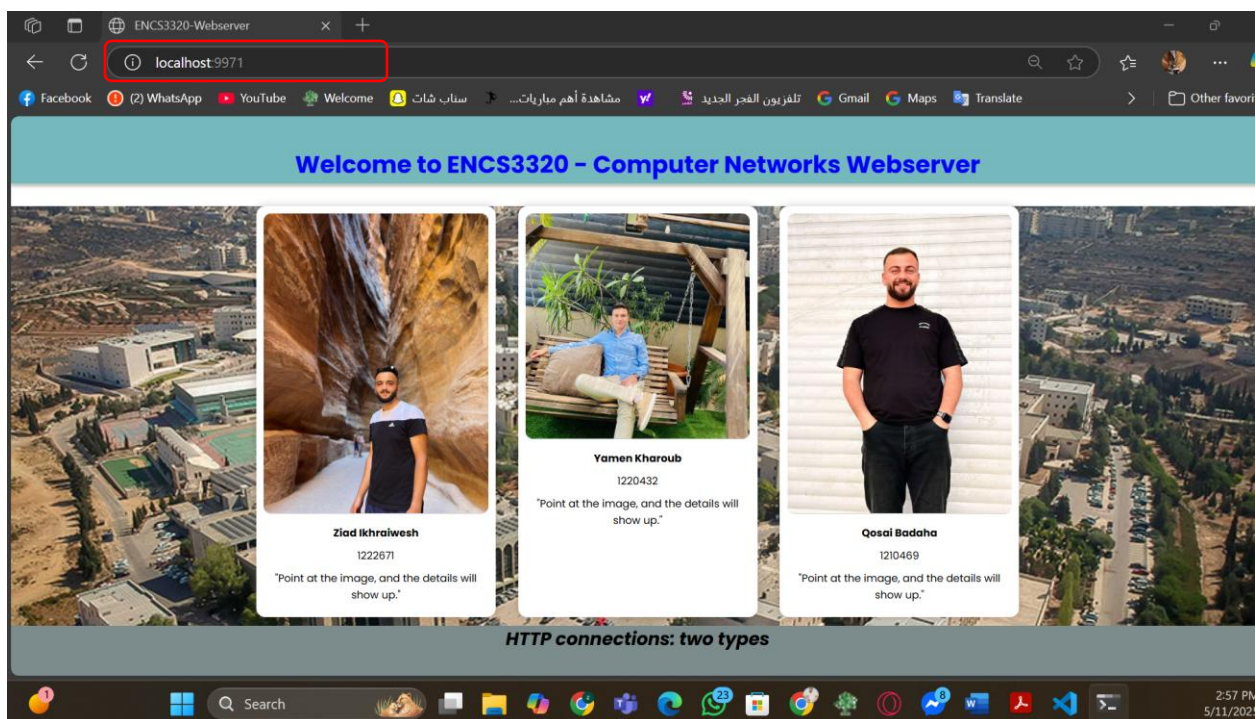


Figure 21: / req

The request '/' opened the main_en.html webpage as requested.

And the browser automatically removes the "/" and sends me to the main page as normal.

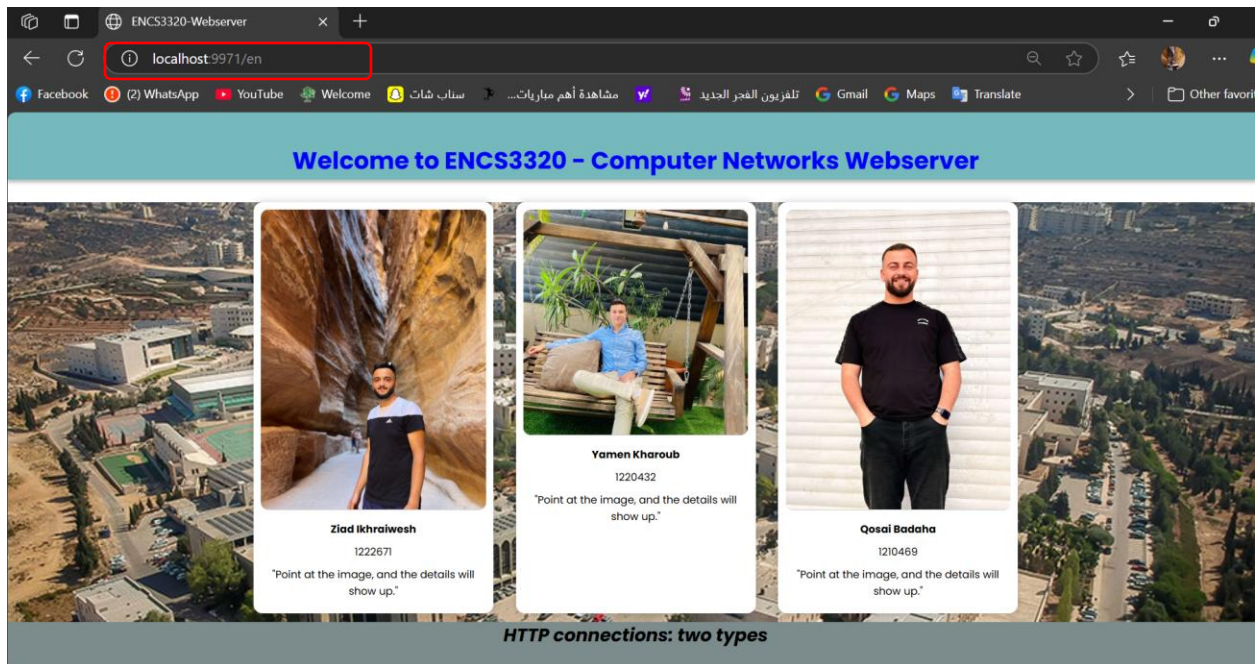


Figure 22:/en req

The request '/en' opened the main_en.html webpage as requested.

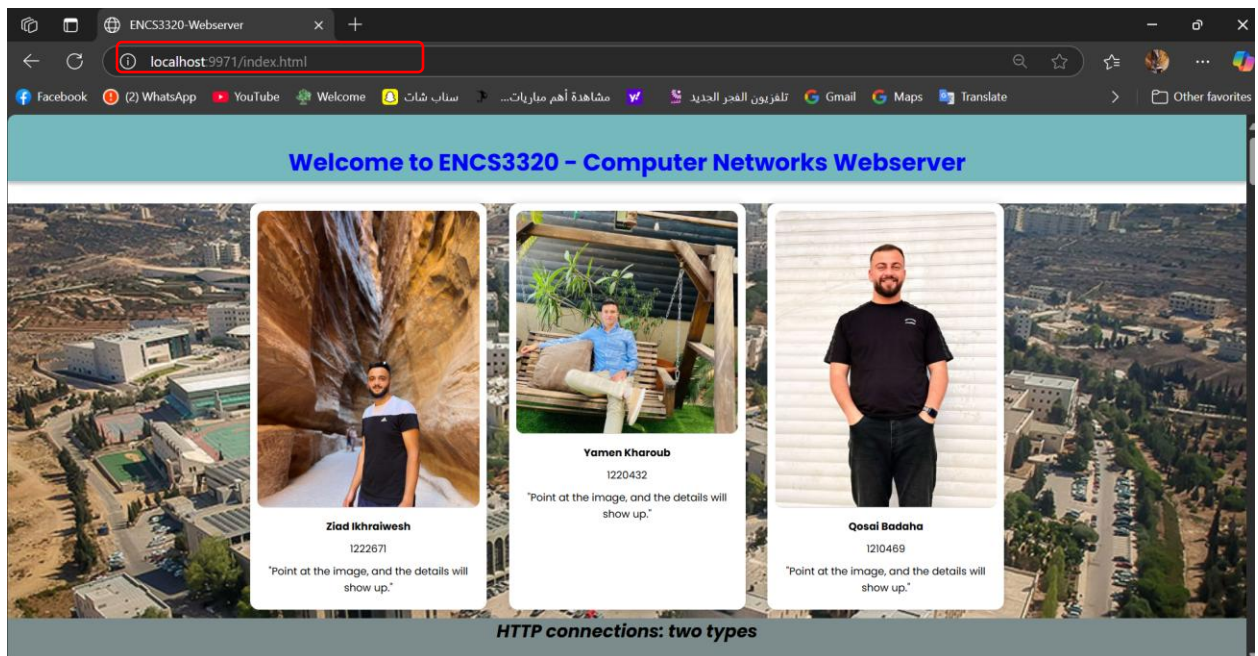


Figure 23: index.html req

The request '/index.html' opened the main_en.html webpage as requested.

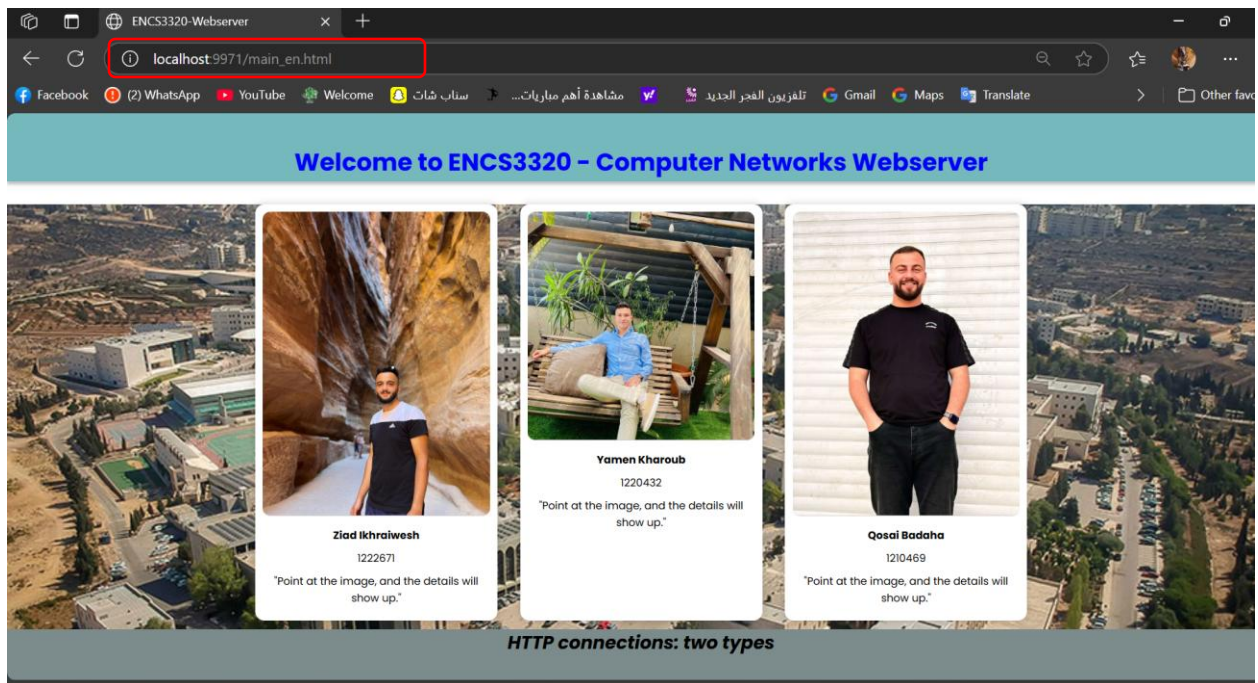


Figure 24: main_en.html req

The request ‘/main_en.html’ opened the main webpage as requested.

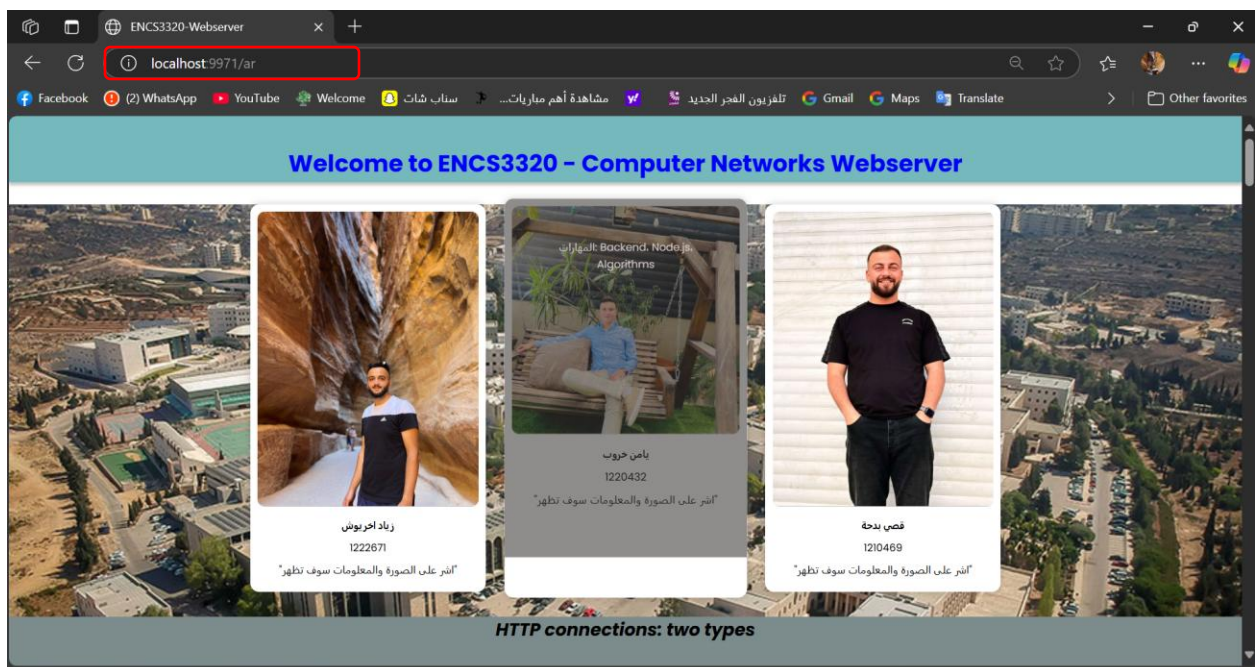


Figure 25: ar req

The request ‘/ar’ opened the main webpage (Arabic version) as requested.

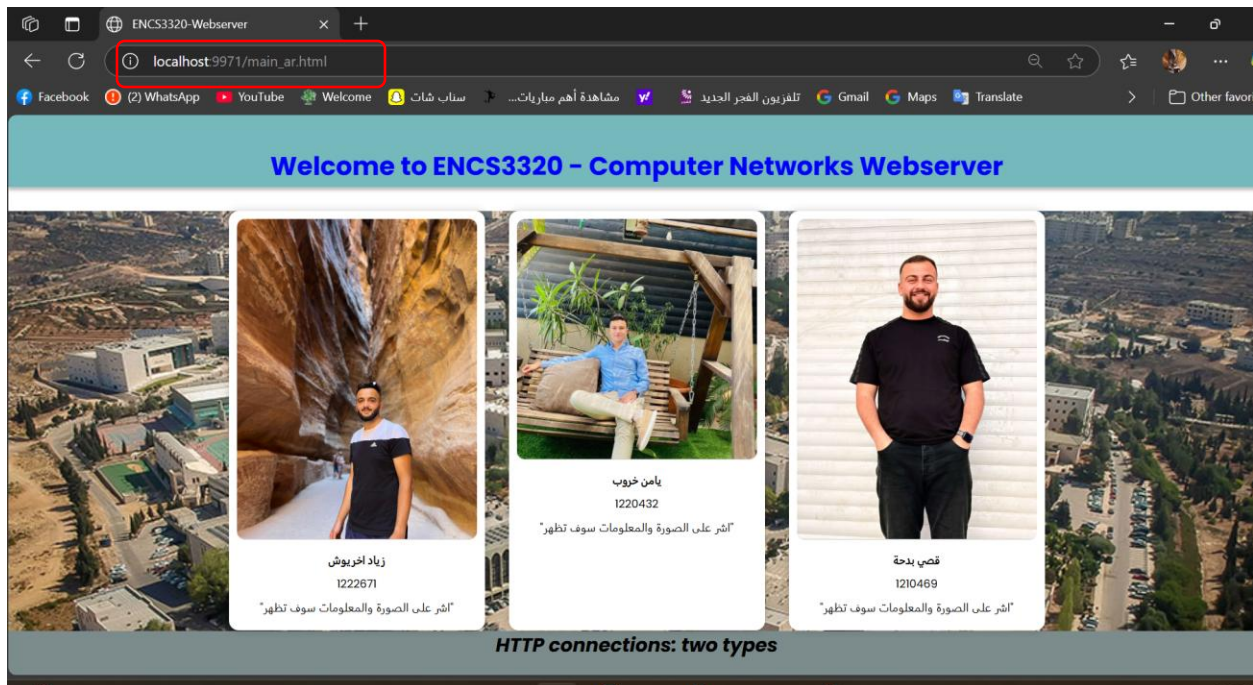


Figure 26: main_ar.html req

The request '/main_ar.html' opened the main webpage (Arabic version) as requested.

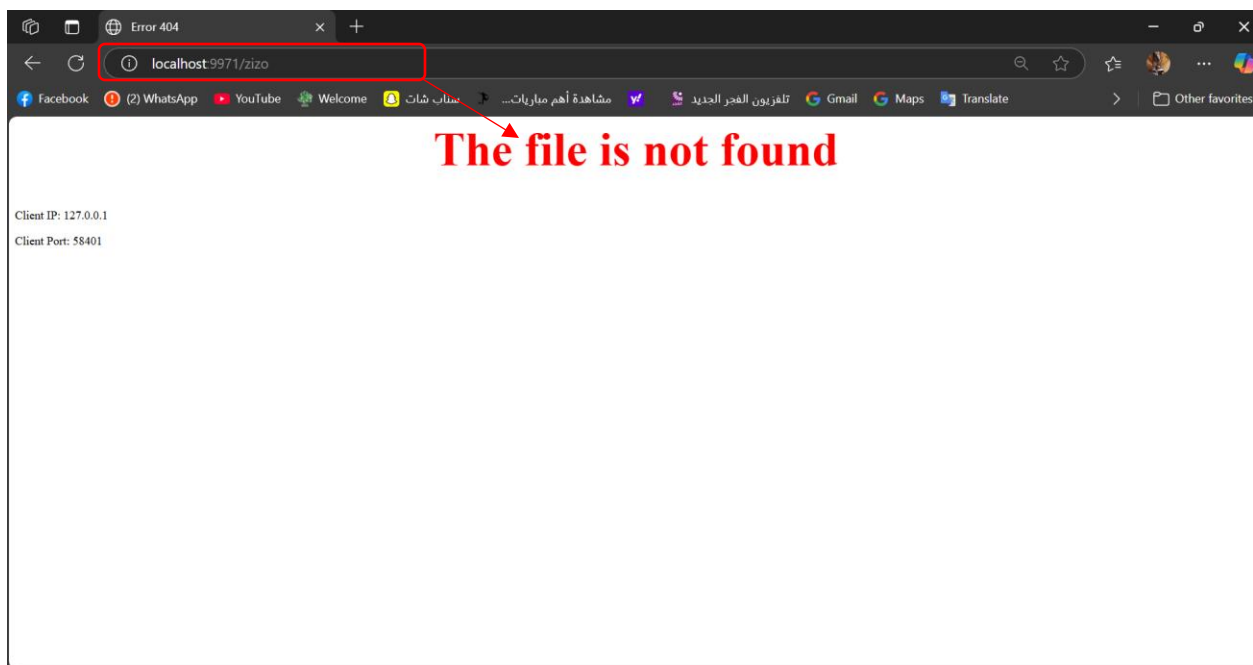


Figure 27: fail request

If your requested was for unfounded file will return the error page.

Now we will discuss the form and server functionality to relate the form html code with the server code we use java script and to create the server code we do this steps in code

first thing we initiate the server:

```
def start_server():
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    host_ip_address = socket.gethostbyname(socket.gethostname())
    port = 9971
    server_socket.bind(("", port))
    server_socket.listen(1)

    print(f"Server started on {host_ip_address}:{port}...")

    while True:
        client_socket, addr = server_socket.accept()
        client_thread = threading.Thread(target=handle_request, args=(client_socket, addr))
        client_thread.start()

if __name__ == "__main__":
    start_server()
```

Figure 28: initiate server

Then we are saving the requested msg and decoded it using utf-8

```
def handle_request(client_socket, addr):
    request_msg = client_socket.recv(1024).decode('utf-8')
    print(f"Request from {addr}: {request_msg}")
    create_response_msg(request_msg, addr, client_socket) # No return value expected
```

Figure 29: handling req msg for the server

After decoded the requested msg we creating decoding msg

And this by see the two things in the req msg the method and the requested file and we determine what the browser must return according to the file bath.

First thing if the file path were an html page that will return all the html and css codes used in this html pages by this code

```
# Function to create response message
def create_response_msg(decoded_msg, addr, client_socket):
    m_type, f_path, _ = decoded_msg.split(" ", 2)
    head = "HTTP/1.1 200 OK \r\n"
    flag = -1
    file = "main_en.html"

    # Handling different paths
    if f_path == "/" or f_path == "/en" or f_path == "/index.html" or f_path == "/main_en.html":
        file = "main_en.html"
        flag = 0
    elif f_path == "/mySite_2671_en.html":
        file = "mySite_2671_en.html"
        flag = 0
    elif f_path == "/ar" or f_path == "/main_ar.html":
        file = "main_ar.html"
        flag = 0
    elif f_path.__contains__(".html"):
        file = f_path
        flag = 0
    elif f_path.__contains__(".css"):
        file = f_path.lstrip("/")
        flag = 4
```

Figure 30: create response msg for the server

In this page the server determines what is the server path and according to it the server return the html and css file requires to this page

According to this code:

```
elif flag == 0:
    client_socket.send(head.encode())
    client_socket.send("Content-Type: text/html; charset=utf-8\r\n".encode())
    client_socket.send("\r\n".encode())
    with open(file, "rb") as f1:
        client_socket.send(f1.read())

elif flag == 4:
    client_socket.send(head.encode())
    client_socket.send("Content-Type: text/css; charset=utf-8\r\n".encode())
    client_socket.send("\r\n".encode())
    with open(file, "rb") as f1:
        client_socket.send(f1.read())
```

And here is an example for requested file msg

When we requested for main_en.html by write this URL in the chrome browser

Localhost: 9971/index.html

```
Request from ('127.0.0.1', 58240): GET /img/instagram.svg HTTP/1.1
Host: localhost:9971
Connection: keep-alive
sec-ch-ua-platform: "Windows"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/136.0.0.0 Safari/537.36 Edg/136.0.0.0
sec-ch-ua: "Chromium";v="136", "Microsoft Edge";v="136", "Not.A/Brand";v="99"
sec-ch-ua-mobile: ?0
Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: no-cors
Sec-Fetch-Dest: image
Referer: http://localhost:9971/index.html
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.9,ar;q=0.8
```

Figure 31 HTTP request

- **The first line in the terminal** shows the IP address and the port number: ('127.0.0.1', 58240)
- **The request line:** GET /img/instagram.svg HTTP/1.1
 - GET: The method used by the client to request a resource from the server.
 - /img/instagram.svg: The URL of the SVG image file requested by the client.
 - HTTP/1.1: The version of the protocol used by the client.
- **The header lines:** the rest of the request message are header lines, each with a header field name and a value. These include:
 - Host: localhost:9971 → indicates the server's host and port.
 - User-Agent, sec-ch-ua, Accept, Referer, etc. → provide client environment information and context for the request.
- **The body:** the body is empty, since GET requests do not carry data in the body.

If we enter name of photo or video in the form html page which is called mySite_2671_en.html that will return the photo or the video if there is founded on the server

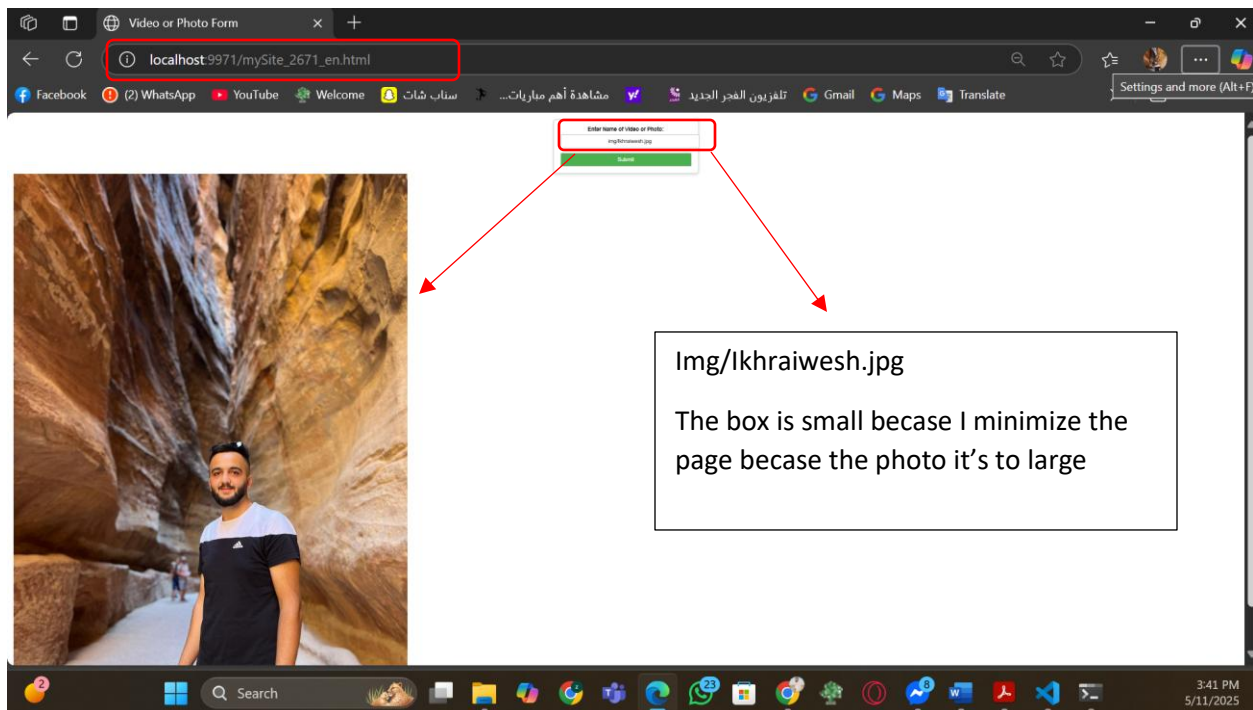


Figure 32: form page functionillity

```
Request from ('127.0.0.1', 58662): GET /img/Ikhraiwesh.jpg HTTP/1.1
Host: localhost:9971
Connection: keep-alive
sec-ch-ua-platform: "Windows"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/136.0.0.0 Safari/537.36 Edg/136.0.0.0
sec-ch-ua: "Chromium";v="136", "Microsoft Edge";v="136", "Not.A/Brand";v="99"
sec-ch-ua-mobile: ?0
Accept: */*
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: http://localhost:9971/mySite_2671_en.html
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.9,ar;q=0.8
```

And here is the HTTP request demonstrate the request msg for Ikraiwesh.jpg By the mySite_2671_en.html page

And if we insert name of photo that did not exist on the server the server will not return the error html page but rather than that it will return the state

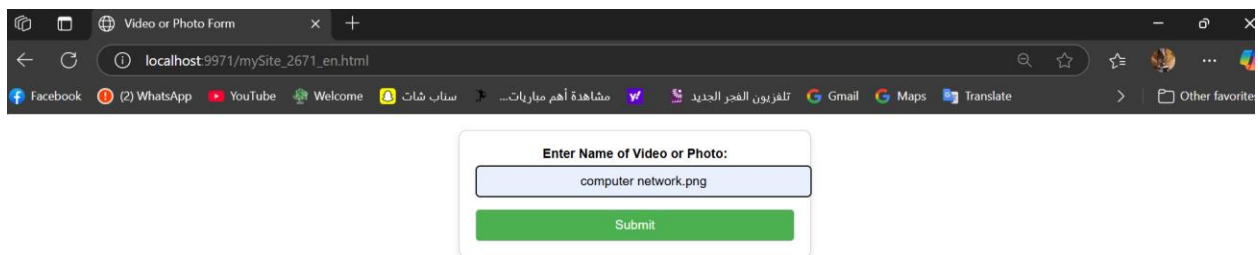
307 Temporary Redirect

And will redirect internally to the google search by this code

```
elif f_path.endswith(".jpg"):
    file = f_path.lstrip("/")
    if not os.path.exists(file): # Check if the image exists
        flag = 5 # Flag 5 for temporary redirect to Google search
    else:
        flag = 2 # Flag for serving the jpg
elif f_path.endswith(".png"):
    file = f_path.lstrip("/")
    if not os.path.exists(file): # Check if the image exists
        flag = 5 # Flag 5 for temporary redirect to Google search

elif flag == 5: # Temporary Redirect for missing photo (search on Google)
    redirect_url = f"https://www.google.com/search?q={f_path[1:]}" # Remove the leading "/" from f_path
    print(f"Redirecting to: {redirect_url}") # Log the redirect URL to ensure it's correct
    client_socket.send("HTTP/1.1 307 Temporary Redirect \r\n".encode())
    client_socket.send(f"Location: {redirect_url}\r\n".encode()) # Set the Location header
    client_socket.send("\r\n".encode()) # End of headers
```

And here is an example



There is no photo called computer network on the device so the server will redirect it to google search

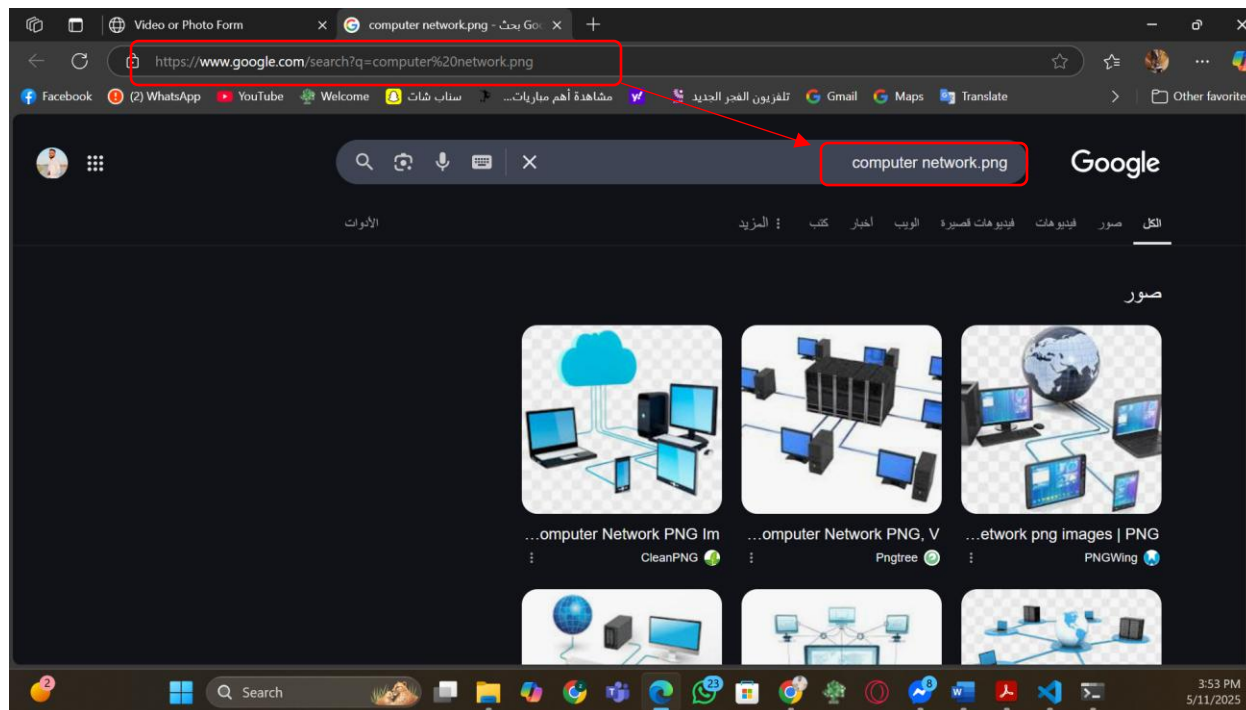


Figure 33: 307 Temporary Redirect

```
Request from ('127.0.0.1', 58821): GET /computer%20network.png HTTP/1.1
Host: localhost:9971
Connection: keep-alive
sec-ch-ua-platform: "Windows"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/136.0.0.0 Safari/537.36 Edg/136.0.0.0
sec-ch-ua: "Chromium";v="136", "Microsoft Edge";v="136", "Not.A/Brand";v="99"
sec-ch-ua-mobile: ?0
Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: no-cors
Sec-Fetch-Dest: image
Referer: http://localhost:9971/mySite_2671_en.html
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: en-US,en;q=0.9,ar;q=0.8

Redirecting to: https://www.google.com/search?q=computer%20network.png
```

Figure 34: Temporary redirect request

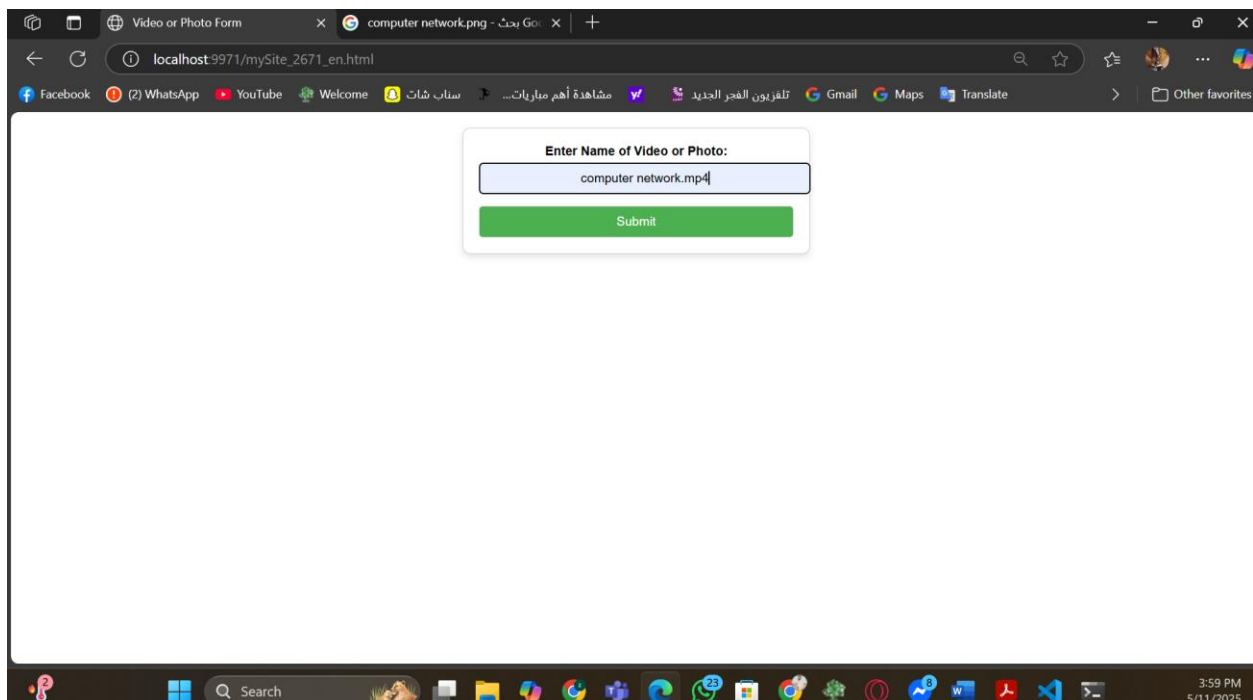
And if we insert name of video that did not exist on the server the server will not return the error html page but rather than that it will return the state

307 Temporary Redirect

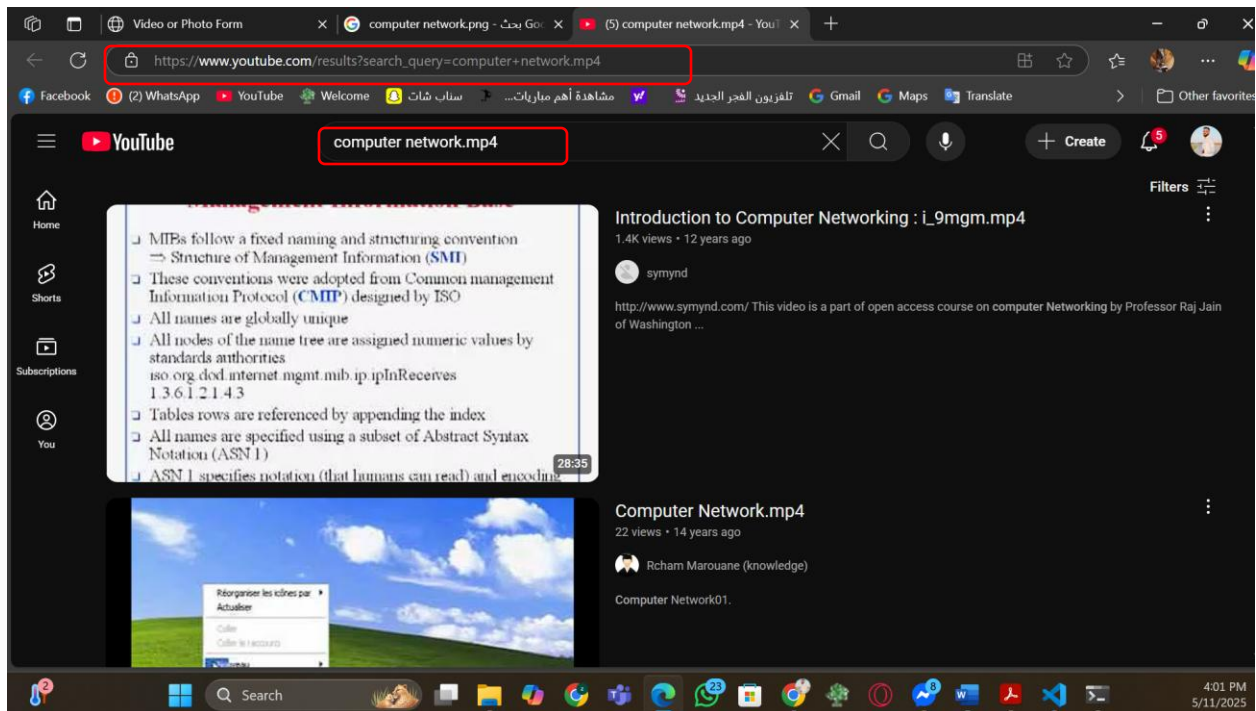
And will redirect internally to the youtube search by this code

```
elif flag == 6: # Temporary Redirect for missing video (search on YouTube)
    head = "HTTP/1.1 307 Temporary Redirect \r\n"
    location = f"Location: https://www.youtube.com/results?search_query={f_path[1:]}\r\n" # Removing the leading '/'
    print(f"Redirecting to: {location}") # Log the redirect URL to ensure it's correct
    client_socket.send(head.encode())
    client_socket.send(location.encode())
    client_socket.send("\r\n".encode())
```

```
elif f_path.endswith(".mp4"):
    file = f_path.lstrip("/")
    if not os.path.exists(file): # Check if the video exists
        flag = 6 # Flag 6 for temporary redirect to YouTube search
    else:
        flag = 3 # Flag for serving the mp4
```



There is no video called computer network on the device so the server will redirect it to YouTube search



```
Request from ('127.0.0.1', 58905): GET /computer%20network.mp4 HTTP/1.1
Host: localhost:9971
Connection: keep-alive
sec-ch-ua-platform: "Windows"
Accept-Encoding: identity;q=1, */q=0
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/136.0.0.0 Safari/537.36 Edg/136.0.0.0
sec-ch-ua: "Chromium";v="136", "Microsoft Edge";v="136", "Not.A/Brand";v="99"
sec-ch-ua-mobile: ?0
Accept: */*
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: no-cors
Sec-Fetch-Dest: video
Referer: http://localhost:9971/mySite_2671_en.html
Accept-Language: en-US,en;q=0.9,ar;q=0.8
Range: bytes=0-

Redirecting to: Location: https://www.youtube.com/results?search_query=computer%20network.mp4
```

Figure 35: Temporary redirect request

We have faced some problem in this task some of these problem is how to call the functions in the server python code after many attempts we decided to call ant turn on it by threads using this code:

```
while True:
    client_socket, addr = server_socket.accept()
    client_thread = threading.Thread(target=handle_request, args=(client_socket, addr))
    client_thread.start()
```

And another problem we faced is how to related between the mySite_2671_en html code and the python code so after we read some resources (you will find it at end of the report)

We finally decided to use java script code it is not the easiest way but we did not find another way

```
29 <script>
30 document.querySelector("form").addEventListener("submit", function(e) {
31     e.preventDefault(); // Prevent form from submitting normally
32
33     var mediaName = document.getElementById("file").value;
34     var image = document.getElementById("image");
35     var video = document.getElementById("video");
36
37     // Hide both image and video initially
38     image.style.display = "none";
39     video.style.display = "none";
40
41     // Check if it's a video or an image
42     if (mediaName.endsWith(".jpg") || mediaName.endsWith(".png")) {
43         image.style.display = "block";
44         image.src = mediaName; // Set image source
45     } else if (mediaName.endsWith(".mp4")) {
46         video.style.display = "block";
47         video.querySelector("source").src = mediaName; // Set video source
48         video.load(); // Load the video
49     } else {
50         alert("Unsupported media type");
51     }
52
53     // Send a request to check if the media exists and handle redirects
54     fetch("/") + mediaName. {
```

```

53 // Send a request to check if the media exists and handle redirects
54 fetch("/") + mediaName, {
55   method: "GET"
56 })
57 .then(response => {
58   if (response.status === 307) {
59     // Redirect automatically if status is 307
60     const redirectUrl = response.headers.get("Location");
61     window.location.href = redirectUrl; // Perform the redirect
62   } else {
63     return response.blob(); // If not a redirect, get the response body
64   }
65 })
66 .then(blob => {
67   if (blob) {
68     // If we have a valid image or video, update the media display
69     const contentType = blob.type;
70     if (contentType.startsWith("image")) {
71       // Image response
72       const objectURL = URL.createObjectURL(blob);
73       image.style.display = "block";
74       image.src = objectURL;
75     } else if (contentType.startsWith("video")) {
76       // Video response

```

```

72       const objectURL = URL.createObjectURL(blob);
73       image.style.display = "block";
74       image.src = objectURL;
75     } else if (contentType.startsWith("video")) {
76       // Video response
77       const objectURL = URL.createObjectURL(blob);
78       video.style.display = "block";
79       video.querySelector("source").src = objectURL;
80       video.load();
81     }
82   }
83 })
84 .catch(error => {
85   console.error("Error:", error);
86 });
87 });
88 </script>

```

Task 3 - TCP/UDP Hybrid Client-Server Game Using Socket Programming

This is a hybrid client-server game using both TCP and UDP socket programming. TCP (port 6000) handles reliable tasks like login and chat, ensuring message delivery. UDP (port 6001) is used for real-time game actions like movement, where speed is prioritized over reliability. The client connects to both ports to communicate with the server. This setup balances reliability and performance.

```
C:\Users\kharo\PycharmProjects\PythonProject1\.venv\Scripts\python.exe C:\Users\kharo\PycharmProjects\PythonProject1\server.py
Server started on <localhost>: TCP 6000, UDP 6001
```

Figure 36

Here the server is runned and waiting for a clients to join the game , by enter their for each client , to join the game carefully .

When the first player join the game:

```
C:\Users\kharo\PycharmProjects\PythonProject1\.venv\Scripts\python.exe C:\Users\kharo\PycharmProjects\PythonProject1\client.py
Please enter your name: yamen
Connected as yamen
UDP connection established

Waiting Room:
yamen
|
```

Figure 37

```
C:\Users\kharo\PycharmProjects\PythonProject1\.venv\Scripts\python.exe C:\Users\kharo\PycharmProjects\PythonProject1\server.py
Server started on <localhost>: TCP 6000, UDP 6001

[NEW CONNECTION] yamen connected from ('127.0.0.1', 50593)
|
```

Figure 38

When the another player join the game:

```
C:\Users\kharo\PycharmProjects\PythonProject1\.venv\Scripts\python.exe C:\Users\kharo\PycharmProjects\PythonProject1\client2.py
Please enter your name: ziad
Connected as ziad
UDP connection established

Waiting Room:
yamen
ziad
```

Figure 39

```
C:\Users\kharo\PycharmProjects\PythonProject1\.venv\Scripts\python.exe C:\Users\kharo\PycharmProjects\PythonProject1\server.py
Server started on <localhost>: TCP 6000, UDP 6001

[NEW CONNECTION] yamen connected from ('127.0.0.1', 50593)

[NEW CONNECTION] ziad connected from ('127.0.0.1', 50613)
Starting game with 2 players.
```

Figure 40

A) Two players register, game starts, one wins:

```
Please enter your name: yamen
Connected as yamen
UDP connection established

Waiting Room:
yamen
Waiting Room:
yamen
ziad
Game started with players: yamen, ziad.
You have 60 seconds to guess the number (1-100)!
Enter your guess (1-100):
55
Feedback: HIGHER!
Enter your guess (1-100):
52
Feedback: LOWER!
Enter your guess (1-100):
54
Feedback: HIGHER!
Enter your guess (1-100):
=== GAME RESULTS ===
Target number was: 53
Winner: ziad
```

Figure 41


```
Please enter your name: ziad
Connected as ziad
UDP connection established

Waiting Room:
yamen
ziad
Game started with players: yamen, ziad.
You have 60 seconds to guess the number (1-100)!
Enter your guess (1-100):
45
Feedback: LOWER!
Enter your guess (1-100):
48
Feedback: LOWER!
Enter your guess (1-100):
53
Feedback: CORRECT!
=== GAME RESULTS ===
Target number was: 53
Winner: ziad
```

Figure 42

```
Server started on <localhost>: TCP 6000, UDP 6001

[NEW CONNECTION] yamen connected from ('127.0.0.1', 50630)
[NEW CONNECTION] ziad connected from ('127.0.0.1', 50631)
Starting game with 2 players.

Game Completed. Winner: ziad
|
```

Figure 43

B) Players guess out of bounds → get warning.

```
Please enter your name: yamen
```

```
Connected as yamen
```

```
UDP connection established
```

```
Waiting Room:
```

```
yamen
```

```
Waiting Room:
```

```
yamen
```

```
ziad
```

```
Game started with players: yamen, ziad.
```

```
You have 60 seconds to guess the number (1-100)!
```

```
Enter your guess (1-100):
```

```
52
```

```
Feedback: HIGHER!
```

```
Enter your guess (1-100):
```

```
110
```

```
WARNING: Out of range, you missed your chance.
```

Figure 44

```
Server started on <localhost>: TCP 6000, UDP 6001
```

```
[NEW CONNECTION] yamen connected from ('127.0.0.1', 50865)
```

```
[NEW CONNECTION] ziad connected from ('127.0.0.1', 50866)
```

```
Starting game with 2 players.
```

```
Send Warning to [yamen]
```

Figure 45

C) One player disconnects mid-game → game continues or ends.

```
Please enter your name: ziad
Connected as ziad
UDP connection established

Waiting Room:
yamen
ziad
Game started with players: yamen, ziad.
You have 60 seconds to guess the number (1-100)!
Enter your guess (1-100):
70
Feedback: HIGHER!
Enter your guess (1-100):
=== GAME RESULTS ===
Target number was: 39
Winner: ziad
|
```

Figure 46

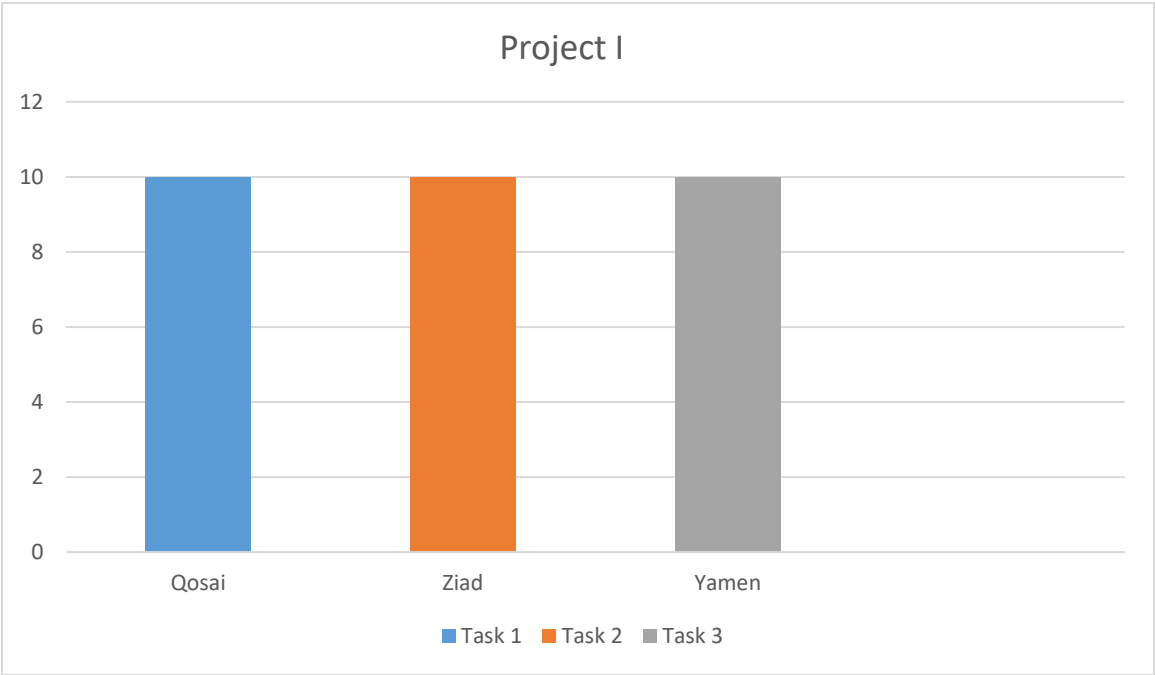
```
Server started on <localhost>: TCP 6000, UDP 6001

[NEW CONNECTION] yamen connected from ('127.0.0.1', 50888)
[NEW CONNECTION] ziad connected from ('127.0.0.1', 50887)
Starting game with 2 players.

[DISCONNECT] yamen disconnected.
Game Completed. Winner: ziad
```

Figure 47

TeamWork:



References:

- [1] https://drive.google.com/drive/folders/1ZTSFxnkihq746bTk9uZVzYVVgxXcIMD?usp=drive_link
- [2] <https://www.naukri.com/code360/library/socket-programming-with-multithreading-in-python>
- [3] <https://eecs485staff.github.io/p4-mapreduce/threads-sockets.html>
- [4] <https://www.youtube.com/watch?v=IEEhzQoKtQU>
- [5] <https://www.youtube.com/watch?v=xceTFWyeag>